

Pro-Talent-Connect

This document defines the data models, REST API endpoints, and role-based access control rules for the Football Player CRM and Scouting Platform. It serves as the single reference for both the development team (API implementation and database schema) and the client (functional clarity and governance).

Purpose:

Platform:	Responsive web-based CRM for football player scouting and data management
Capacity:	Up to 150 player profiles
Audience:	Football scouts, coaches, administrators Ages 18–40 Low-medium technical knowledge
Concurrency:	60–100 simultaneous users
Infrastructure:	Low-cost cloud hosting, automatic backups, zero-install deployment

User Roles & Permissions:

Super Admin:

- Full control over all platform data, content, and user management. Typically the head coach or organisation owner.
- Full CRUD (Create / Read / Update / Delete) on Player profiles
- Full CRUD on Blog & Article posts
- Full CRUD on About Me / Organisation content
- Create, update, and demote Admin accounts
- System-level authority - only role with delete access

Admin (Scout / Assistant):

- Operational access for day-to-day scouting activities. No delete or user-management capabilities.
- Create, View, and Update Player profiles
- Create, View, and Update Blog & Article posts
- View About Me / Organisation content
- No delete access on any module
- No admin management capabilities

Players Module

FIELD	DATA TYPE	DESCRIPTION
id	UUID	Unique player identifier (primary key)
name	String	Full name of the player
age	Integer	Age in years
age_group	String	Age category (e.g. U13, U15, U17, U19, Senior)
gender	Enum	Male / Female
position	String	Playing position (e.g. Striker, Midfielder, Goalkeeper)
nationality	String	Player nationality / country
height	Float	Height in centimetres
weight	Float	Weight in kilograms
jersey_no	Integer	Assigned jersey number
size	String	Jersey size (XS / S / M / L / XL / XXL)
city	String	Player's city of residence

address	String	Full address
contact_phone	String	Contact phone number
contact_email	String	Contact email address
profile_image	String (URL)	Profile photo -stored in cloud image storage
scouting_notes	Text	Scout observations and performance notes
career_history	Text	Previous clubs, academies, career timeline
media_links	Array<String>	URLs to videos, highlight reels, documents
created_at	DateTime	Record creation timestamp (ISO-8601)
updated_at	DateTime	Last update timestamp (ISO-8601)

REST API Endpoints

- GET /api/players - List all players (filterable by age_group, position, gender)
- POST /api/players - Create a new player profile - Admin / Super Admin
- GET /api/players/{id} - Retrieve a single player profile
- PUT /api/players/{id} -Update player profile - Admin / Super Admin
- DELETE /api/players/{id} -Delete player - Super Admin only
- GET /api/players/search -Search players by name, city, age group, or position

Blogs & Articles Module

FIELD	DATA TYPE	DESCRIPTION
id	UUID	Unique article identifier (primary key)
title	String	Article or blog post title
slug	String	URL-friendly title identifier (auto-generated)
content	Text (HTML)	Full article body content (rich text / HTML)
excerpt	String	Short preview / summary (max 300 characters)
author_id	UUID (FK)	Foreign key referencing users.id
cover_image	String (URL)	Cover image URL -stored in cloud image storage
tags	Array<String>	Topic tags (e.g. 'scouting', 'transfers', 'U17')
status	Enum	DRAFT / PUBLISHED / ARCHIVED
published_at	DateTime	Publish timestamp (ISO-8601); null if draft
created_at	DateTime	Record creation timestamp (ISO-8601)
updated_at	DateTime	Last update timestamp (ISO-8601)

REST API Endpoints

- GET /api/blogs -List all published articles (public-facing)
- GET /api/blogs/all -List all articles including drafts - Admin / Super Admin
- POST /api/blogs -Create a new article - Admin / Super Admin
- GET /api/blogs/{id} -Retrieve a single article by ID or slug
- PUT /api/blogs/{id} -Update article content or status - Admin / Super Admin
- DELETE /api/blogs/{id} -Delete article permanently - Super Admin only
- PATCH /api/blogs/{id}/publish -Toggle article status to PUBLISHED - Admin / Super Admin

About Me / Organisation Module

FIELD	DATA TYPE	DESCRIPTION
id	UUID	Singleton record identifier
org_name	String	Organisation / academy name
mission	Text	Mission statement
history	Text	Background and history of the organisation
credentials	Text	Achievements, affiliations, certifications
contact_email	String	Public contact email address
contact_phone	String	Public contact phone number
location	String	City / region
images	Array<String>	Gallery image URLs -stored in cloud image storage
social_links	JSON	Object with keys: instagram, facebook, twitter, youtube
updated_at	DateTime	Last update timestamp (ISO-8601)

REST API Endpoints

- GET /api/about -Retrieve organisation profile (public-facing)
- PUT /api/about -Update organisation profile - Super Admin only
- POST /api/about/images -Upload new gallery images - Super Admin only

User & Role Module

FIELD	DATA TYPE	DESCRIPTION
id	UUID	Unique user identifier (primary key)
name	String	Full name of the administrator
email	String	Unique email address -used for login
password	String (hashed)	bcrypt-hashed -never store plain text
role	Enum	SUPER_ADMIN / ADMIN
profile_image	String (URL)	Profile picture URL
is_active	Boolean	Account active status (true / false)
last_login	DateTime	Timestamp of most recent login (ISO-8601)
created_at	DateTime	Account creation timestamp (ISO-8601)

Authentication Endpoints

- POST /api/auth/login -Login with email + password → returns JWT token
- POST /api/auth/logout -Invalidate current session token
- POST /api/auth/refresh -Refresh expired JWT token
- Admin Management Endpoints
- GET /api/admins -List all admin accounts - Super Admin only
- POST /api/admins -Create a new Admin account - Super Admin only
- GET /api/admins/{id} -View admin profile - Super Admin only
- PUT /api/admins/{id} -Update admin details - Super Admin only
- PATCH /api/admins/{id}/demote -Demote an Admin / deactivate account - Super Admin only
- DELETE /api/admins/{id} -Delete an Admin account - Super Admin only

Security & Non-Functional Notes

Authentication:	JWT (JSON Web Token) -issued on login, required on all protected endpoints
Authorisation:	Role-based middleware enforced server-side on every request
Password storage:	bcrypt hashing -plain text passwords are never stored
HTTPS:	All traffic must be served over TLS/HTTPS
CORS:	Restrict allowed origins to the deployed frontend domain
Load capacity:	Basic handling for 60–100 concurrent users
Backups:	Automatic cloud backups -no manual intervention required
Action tracking:	All create / update / delete actions logged with user ID and timestamp